

Policy-as-Code vs Ad-Hoc Scripting for PD Log Forensics

White paper for the `vortex-vs-adhoc` **project**

Subtitle: A 1:1 comparison of Vortex-style YAML `search_by_rule` against Bash, Python, and Perl implementations of the same seven design-health rules.

Companion code: `vortex-vs-adhoc/` in this repository

Measured artifacts: `results/SUMMARY.txt`, `results/loc.txt`

Abstract

Physical-design and signoff organizations drown in multi-gigabyte tool logs. Teams respond with personal grep pipelines, Bash helpers, Python one-offs, and Perl holdovers. Those scripts work — until the rule set grows, the author leaves, or front-end / back-end / CAD need a **shared** definition of “what is a violation.”

This paper compares two ways to encode the **same** seven design-health rules:

1. **YAML DSL (policy-as-code)** — declarative intent (`policy/design_health.yaml`), executed by a reference runner or by the Vortex product binary.
2. **Ad-hoc procedural scripts** — Bash, Python, and Perl that embed patterns and thresholds in code.

On a public synthetic PD-style log (**200 k lines**, ~5.9 MB), all four engines produce **identical hit counts** and the **same triggered-rule set**. Non-comment LOC for the YAML policy is **51** versus **96 / 123 / 69** for Bash / Python / Perl. The methodological claim is not “YAML is always faster than grep on a 6 MB file.” The claim is that **intent belongs in a shared, reviewable policy**, while engines (Vortex, or a thin runner) own implementation, performance, and audit.

Table of contents

1. [Motivation](#)
2. [Problem statement](#)
3. [Experimental design](#)
4. [The seven-rule policy](#)
5. [Ad-hoc counterparts](#)
6. [Measured results](#)
7. [Dimension-by-dimension analysis](#)
8. [Where Vortex fits](#)
9. [How to reproduce on your machine](#)
10. [Limitations](#)
11. [Conclusion](#)

12. [Appendix A — raw SUMMARY](#)
13. [Appendix B — glossary](#)

1. Motivation

Farm reality:

- Chamber logs are huge; humans cannot read them linearly.
- “Just grep it” becomes a private dialect per engineer.
- Methodology leads cannot review a 200-line Bash regex nest in a design review.
- AI / RAG workflows need a **retrieval front-end** that returns named, structured findings — not an unstructured grep firehose.

Organizations therefore need a **shared language** for log forensics: named rules, severities, thresholds, and stable output. That language is policy-as-code. The scanner that executes it at GB scale (Vortex) is the product engine.

2. Problem statement

Given seven design-health rules, compare (a) encoding them as a YAML DSL versus (b) encoding them as ad-hoc Bash / Python / Perl, on **correctness**, **LOC**, **editability**, **team dynamics**, and **runtime** on a controlled synthetic log — and document how anyone can reproduce the experiment locally.

Success criteria:

1. Functional equivalence — same triggers and counts across engines.
2. Transparent LOC and timing numbers checked into results/.
3. Clear separation: YAML = intent; runner/Vortex = engine; ad-hoc = intent⊕implementation fused.

3. Experimental design

Item	Choice
Policy	policy/design_health.yaml (7 rules)
Engines	YAML reference runner, Bash, Python, Perl; optional VORTEX_BIN
Workload	scripts/gen_synthetic_log.py → 200 000 lines, fixed seed
LOC metric	Non-blank, non-full-line-comment lines
Harness	examples/compare_all.sh
Agreement	Triggered-rule set + per-rule hit counts

Fairness notes:

- Patterns are intentionally aligned across YAML and ad-hoc ports.

- Bash uses `grep -Eci` (fast C path). Python/Perl/YAML-runner use interpreted loops — expected slower on small files.
- Synthetic log is **PD-shaped**, not a customer proprietary chamber dump.

4. The seven-rule policy

Rules (names):

1. `setup_violation` (error, threshold 0)
2. `hold_violation` (error, threshold 0)
3. `max_transition` (warning, threshold 10)
4. `drc_error` (error, threshold 0)
5. `antenna` (warning, threshold 5)
6. `congestion_hotspot` (warning, threshold 0)
7. `fatal_or_abort` (fatal, threshold 0)

The YAML is self-documenting: each rule carries severity, pattern, `count_threshold`, `context_lines`, and a human note. Editing `count_threshold: 10 → 20` is a one-field change.

4.1 Side-by-side: one rule as intent vs implementation

YAML (intent — reviewable by a methodology lead):

```
- name: max_transition
  severity: warning
  pattern: 'max_transition|MaxTran|MAXTRAN'
  count_threshold: 10
  context_lines: 1
  note: Flag only if more than 10 MaxTran hits (noise floor).
```

Bash (implementation — policy buried in procedure):

```
PAT_MAXTRAN='max_transition|MaxTran|MAXTRAN'
THR_MAXTRAN=10
count=$(grep -Eci -- "$PAT_MAXTRAN" "$LOG" || true)
triggered=0; (( count > THR_MAXTRAN )) && triggered=1
# ... plus JSON emission, context printing, severity strings ...
```

The Bash fragment is only the **core** of one rule. The full script also reinvents reporting, thresholds for six other rules, and exit semantics. Change the noise floor in YAML: one field. Change it in Bash + Python + Perl: three files, three dialects, three test paths.

4.2 Edit-cost walkthrough (measured in engineer minutes)

Change	YAML	Ad-hoc (×3 languages)
Raise MaxTran threshold 10→20	Open YAML, edit one integer (~5 s)	Find <code>THR_MAXTRAN</code> / <code>dict</code> / <code>\$thr</code> , edit each port, re-run (~2–5 min)
Add <code>lvs_error</code> rule		

Change	YAML	Ad-hoc (×3 languages)
	New named block (~1 min)	Copy block in Bash, Python, Perl; align patterns; retest (~15–30 min)
Rename severity warning→error for antenna	One field	Hunt string literals / enums in each script

5. Ad-hoc counterparts

Script	Character
adhoc/bash_search_by_rule.sh	Procedural blocks; thresholds as shell vars; JSON side-dump reinvented
adhoc/python_search_by_rule.py	Cleaner than Bash, still a code-owned rule table
adhoc/perl_search_by_rule.pl	Classic EDA-scripting lineage; patterns in code

Adding an eighth rule means copying a procedural block and hoping thresholds stay consistent with docs and sibling languages.

N-language tax: Once a company has Bash and Python and Perl holdovers (common in EDA), every policy change is multiplied. YAML collapses that to one file; engines consume it.

6. Measured results

Host run embedded in this paper’s results/SUMMARY.txt (re-run to refresh).

6.1 Lines of code

Artifact	Non-comment LOC	Raw lines
YAML policy	51	67
Bash ad-hoc	96	133
Python ad-hoc	123	138
Perl ad-hoc	69	77
YAML reference runner	104	—

Bash / YAML $\approx 1.88\times$ for the same seven rules (implementation LOC vs intent LOC).

6.2 Correctness

Triggered set: all seven rules fire on the synthetic log.

MATCH across yaml_dsl, bash, python, perl.

Per-rule counts identical (e.g. setup 52, hold 66, max_transition 57, ...).

6.3 Wall time (200 k-line synthetic log)

Engine	Time (s)
bash	~0.23
yaml_dsl reference	~0.96
python	~0.98
perl	~1.03
vortex	skip unless <i>VORTEX_BIN</i> set

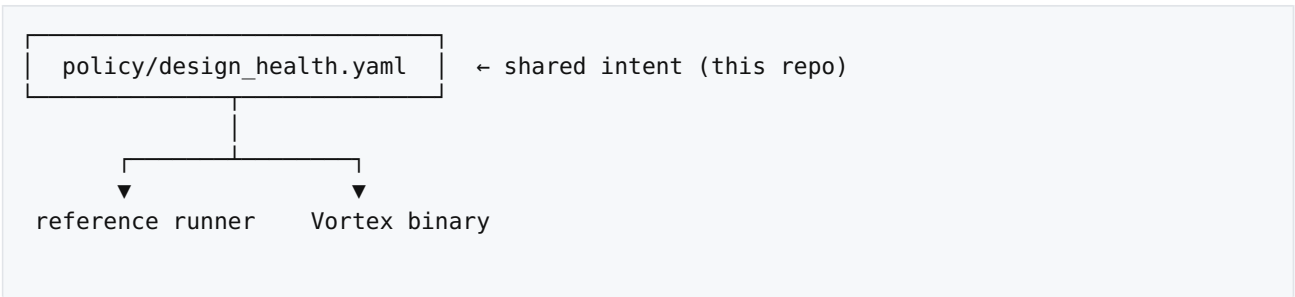
Interpretation: on this size, grep wins the microbench. The paper’s primary claim remains **maintainability and alignment**. Performance leadership on multi-GB chamber logs is a **Vortex engine** claim — measure with *VORTEX_BIN* and a real log; do not over-read the 6 MB synthetic timing.

7. Dimension-by-dimension analysis

Dimension	YAML DSL	Ad-hoc Bash/Grep (and siblings)
Code & maintainability	~51 LOC intent; localized edits	~96-123 LOC; risky cross-edits
Readability	Declarative	Procedural + brittle regex
Edit a rule	Seconds	Minutes
Add a rule	New named block	Copy/modify/test
Who can write?	List-literate engineers	Language + regex experts
Who can review?	Broad team / leads	Experts only
Alignment	Shared policy-as-code	Fragmentation / scripting debt
Output	Structured severities & counts	Raw / bespoke
Audit trail	Product-native (Vortex)	DIY
Philosophy	Intent	Implementation

Killer insight: The value of the YAML surface is a **shared, verifiable language** that aligns FE, BE, and CAD. The cost of ad-hoc scripts is maintenance, review risk, and cognitive load — even when grep is fast.

8. Where Vortex fits



```
(demo / CI)          (product: GB perf,  
                      audit, distribution)
```

- This repository teaches and measures the **policy vs ad-hoc** gap.
- Vortex is the **engine** customers install on-prem (sub-1 MB class binary, GB-scale search).
- LLMs / RAG sit **above** Vortex: the binary is a retrieval/compression front-end that feeds the right slices into a fixed context window.

9. How to reproduce on your machine

```
git clone <this-repo>  
cd vortex-vs-adhoc  
pip install pyyaml  
  
./examples/compare_all.sh  
cat results/SUMMARY.txt  
  
# optional product path  
export VORTEX_BIN=/path/to/vortex  
./examples/compare_all.sh  
  
# stress the log size  
LINES=1000000 REGEN_LOG=1 ./examples/compare_all.sh
```

Requirements: bash, python3, perl, grep, pip install pyyaml.

10. Limitations

- Synthetic log $\neq$ customer chamber log.
- Reference runner $\neq$ Vortex performance.
- LOC metrics depend on comment style; we publish the exact counter (`grep -cvE '^s*(#|/|/|$)'`).
- Timing variance on shared CI hosts — treat as order-of-magnitude.

11. Conclusion

For the same seven design-health rules, a **51-line YAML policy** matches **Bash / Python / Perl** on correctness while keeping intent reviewable by non-scripting stakeholders. Ad-hoc scripts fuse intent into implementation and create scripting debt. Vortex industrializes the DSL with performance and audit; this repo makes the comparison **reproducible on any laptop**.

Appendix A — raw SUMMARY

See checked-in [results/SUMMARY.txt](#) from the harness run that accompanied this paper.

Appendix B — glossary

Term	Meaning
Policy-as-code	Rules stored as data (YAML), not buried in scripts
Ad-hoc script	Procedural scanner owned by one engineer's dialect
DSL	Domain-specific language (<code>search_by_rule</code> surface)
Triggered	<code>count > count_threshold</code>
Vortex	On-prem PD log forensics engine (product; not shipped in this folder)